

Error States: UI Uplift / Strings Only

Five core workflow commands, audited against the existing error infrastructure.

Moderne CLI — April 2026 | F19 focus | companion to Annie Rimbach's Quick Wins frame

Five core workflow commands, from the user's side.

One command already writes a useful failure message. Three end at a dead end — "Try: Report to support@moderne.io." One hides the failure inside a "partially succeeded" banner. These frames are proposals for what the user sees when things go wrong — the text on screen, the structure of the message, and whether the user knows what to do next.

● 1 useful failure message (mod run) | ● 3 dead-end failure messages (build, study, git sync) | ● 1 zero-result that's easy to miss (search) | ⚡ Message content only

EXTEND mod run — the pattern that works

Run already does it right. Extend the forward-chain on success.

Current output

```
$ mod run . --  
recipe=org.openrewrite.java.cleanup.UnnecessaryParentheses
```

- Loading recipe
- Running recipe on 47 repositories
 - ✓ 42 repositories modified
 - ✓ 5 unchanged
- What to do next:
Use `mod study --last-recipe-run` to view results.

```
MOD SUCCEEDED in (3m 24s)
```

Proposed (Primer-aligned)

```
$ mod run . --  
recipe=org.openrewrite.java.cleanup.UnnecessaryParentheses
```

- Loading recipe
- Running recipe on 47 repositories
 - ✓ 42 repositories modified
 - ✓ 5 unchanged

WHAT TO DO NEXT

- ▶ `mod study --last-recipe-run` — View results by repo.
- ▶ `mod git commit --last-recipe-run` — Commit changes across repos.
- ▶ `mod git push --last-recipe-run` — Push to remotes.

```
MOD SUCCEEDED in (3m 24s)
```

WHAT THIS CARD SHOWS

mod run is the most helpful of the five. When the recipe name can't be found, the error already tells the user two concrete next steps — search for recipes, or export the full list. After success, the CLI already prints a "what to do next" line.

The frame on the right pushes that a little further: instead of one hint, show the common paths forward — view results, commit, push — so users don't have to guess which one comes first.

This card is the benchmark for the rest. The other four could feel like this without changing what commands do — only what their messages say.

HOLLOW TRY mod build — hollow Try

Most common failure: no build tool detected in path.

Current output

```
$ mod build .  
  
FAILURE: mod failed with an exception  
  
● What went wrong:  
No build tool found for directory /home/user/project  
  
● Try:  
► Report to support@moderne.io  
  
MOD FAILED in (2s)
```

Proposed (Primer-aligned)

```
$ mod build .  
  
FAILURE: mod failed with an exception  
  
● WHAT WENT WRONG  
No build tool found in /home/user/project.  
  
? Hint: Moderne auto-detects pom.xml, build.gradle, setup.py,  
package.json, and others at the project root.  
  
● TRY  
► Add a build config – pom.xml, build.gradle, or setup.py.  
► Already have one? Point the CLI at it:  
  mod build <path> --only-tool gradle  
► Still stuck? Report to support@moderne.io  
  
MOD FAILED in (2s)
```

WHAT THIS CARD SHOWS

Today's failure message ends at "Report to support." For a user who landed here because they ran mod build in the wrong directory, or whose project uses a build tool the CLI didn't auto-detect, there's no path out except email.

The frame on the right keeps the same four-section structure but fills the Try with two concrete recoveries the user can actually attempt. Support stays in the list — demoted to gray, kept as the last option. This preserves the escalation path without leading with it.

Scope: message content only. The existing Try section gets new bullets; nothing else about the error template changes.

HOLLOW TRY mod study — vague Try

Most common failure: no matching data table in last recipe run.

 Current output

```
$ mod study . --last-recipe-run --data-table=RewriteSources
```

FAILURE: mod failed with an exception

- What went wrong:
No data tables available
- Try:
 - ▶ Double-check that the data table name was spelled correctly or that the recipe produced results.
 - ▶ Report to support@moderne.io

MOD FAILED in (1s)

 Proposed (Primer-aligned)

```
$ mod study . --last-recipe-run --data-table=RewriteSources
```

FAILURE: mod failed with an exception

- WHAT WENT WRONG
No data table matching "RewriteSources" in the last recipe run.

? **Hint:** The last run produced 0 data tables. The recipe may not emit tables, or the run failed before any were written.

- TRY
 - ▶ Run a recipe that emits data first:
 mod run <path> --recipe=<RecipeName>
 - ▶ Double-check the table name — names are case-sensitive.
 - ▶ Still stuck? Report to support@moderne.io

MOD FAILED in (1s)

WHAT THIS CARD SHOWS

The current Try compresses three distinct causes — wrong name, recipe didn't emit data, run never finished — into one sentence that ends with "or that the recipe produced results." That's less a suggestion than a shrug.

The frame on the right splits those causes apart and offers the one action that fixes most of them: run a recipe first. The case-sensitivity note catches typos. The support line stays as the last option.

Scope: message content only. The Try bullets and the hint line are text inside sections that already exist.

HOLLOW TRY mod git sync — hollow Try

Most common failure: organization name doesn't match a configured org.

 Current output

```
$ mod git sync moderne . --organization=MyOrg
```

```
FAILURE: mod failed with an exception
```

- What went wrong:
No organization found matching this name
- Try:
 - ▶ Report to support@moderne.io

```
MOD FAILED in (3s)
```

 Proposed (Primer-aligned)

```
$ mod git sync moderne . --organization=MyOrg
```

```
FAILURE: mod failed with an exception
```

- **WHAT WENT WRONG**
Unable to sync "MyOrg" from Moderne.

? Hint: A few things can cause this — the org name may not match, your login may have expired, or the CLI may not be reaching Moderne right now.

- **TRY**
 - ▶ Check your connection and org list:
mod config moderne show
 - ▶ Refresh your login:
mod config moderne login
 - ▶ Still stuck? Report to support@moderne.io

```
MOD FAILED in (3s)
```

WHAT THIS CARD SHOWS

Today this error can fire for several different reasons — the org name doesn't exist, the user's auth has expired, their network can't reach Moderne, or there's an SSL issue — and the current message looks the same in most of them.

Because the CLI today can't always tell these apart at the moment this message is shown, the frame on the right avoids naming a specific cause. The hint acknowledges the ambiguity. The Try list offers two diagnostic commands that help regardless of which cause is the real one — **show** lists configured orgs and surfaces connectivity; **login** refreshes auth.

Scope: message content only. If the CLI later distinguishes these failure modes, the message and Try can be made more specific per cause.

EASY TO MISS mod search — quiet zero-result

0 of 47 repos searched is easy to miss when "partially succeeded" reads as mostly OK.

●●● Current output

```
$ mod search . "usages of FindMethods"

● Searching 47 repositories
  ▲ No search index on 47 repositories (skipped)

MOD PARTIALLY SUCCEEDED in (3s)
▶ 0 of 47 repositories searched
```

●●● Proposed (Primer-aligned)

```
$ mod search . "usages of FindMethods"

● Searching 47 repositories
  ▲ 0 repositories searched — all 47 skipped (no search index).
    Run mod postbuild search index <path> to build indexes
    from existing LSTs, then re-run this search.

MOD PARTIALLY SUCCEEDED in (3s)
▶ 0 of 47 repositories searched
```

WHAT THIS CARD SHOWS

Scope: message content only. The completion banner stays **MOD PARTIALLY SUCCEEDED** — no change to exit codes or classification. The proposal only sharpens the message that prints above it.

Today the skip line reads "▲ No search index on 47 repositories (skipped)", which is easy to scan past. The revision leads with **0 repositories searched** so the user immediately sees that nothing ran, and inlines the fix command (**mod postbuild search index <path>**) right next to it — the CLI already prints this per-skipped-repo, so surfacing it once here is wording, not new behavior.

Whether an all-skip result *should* read as partial success or outright failure is a product call, not a UX one — flagging as an observation for the team.

WHAT THE USER SEES TODAY

Command	Current failure message	Gap from the user's side
mod run	Two concrete suggestions + support fallback	None — this is the working example.
mod build	"Try: Report to support@moderne.io"	Dead end. No path forward for a user whose project isn't auto-detected.
mod study	"Try: check spelling or recipe produced results"	Vague. Three distinct causes compressed into one sentence.

Command	Current failure message	Gap from the user's side
<code>mod git sync</code>	"Try: Report to support@moderne.io "	Dead end — and multiple distinct causes may look identical today.
<code>mod search</code>	"MOD PARTIALLY SUCCEEDED — 0 of 47 searched"	Skip reason is above the banner but easy to scan past; "partially succeeded" reads as mostly OK.

Prepared by Jayd Jackson, UX | April 2026 | Primer CLI Design Guidelines applied | Companion to Annie Rimbach's Quick Wins frame